WEEK - 8

Java Programs on Inheritance, super Keyword,

Method Overriding and Dynamic Method Dispatch

Name: Chandan

B.Tech CSE

Question 1: Single Inheritance

1. Question

Write a Java program to demonstrate single inheritance by creating a Person superclass containing the name and age of a person. Create a Student subclass containing roll number and branch. Inherit the properties of Person and display the complete student details.

2. Program

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

class Student extends Person {
    int rollNo;
    String branch;

    Student(String name, int age, int rollNo, String branch) {
        super(name, age);
        this.rollNo = rollNo;
        this.branch = branch;
    }

    void display() {
        System.out.println("Name   : " + name);
        System.out.println("Age    : " + age);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Branch : " + branch);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Chandan", 19, 101, "CSE");
        s.display();
    }
}
```

3. Output

```
Name   : Chandan
Age    : 19
Roll No: 101
Branch : CSE
```

Question 2: Using super Keyword

1. Question

Write a Java program to demonstrate the use of the super keyword. Create a Vehicle superclass with a speed variable and a display() method. Create a Car subclass with a model variable and override the display() method. Use super to access the superclass variable and method.

2. Program

```
class Vehicle {
    int speed = 80;

    void display() {
        System.out.println("Vehicle speed: " + speed);
    }
}

class Car extends Vehicle {
    String model = "Swift";

    void display() {
        System.out.println("Super class speed: " + super.speed);
        super.display();
        System.out.println("Car model: " + model);
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.display();
    }
}
```

3. Output

```
Super class speed: 80
Vehicle speed: 80
Car model: Swift
```

Question 3: Multilevel Inheritance

1. Question

Write a Java program to demonstrate multilevel inheritance by creating three classes: Person, Employee, and Manager. The Person class should contain personal details, Employee should contain employee details, and Manager should contain department details. Display all the details using inherited properties.

2. Program

```
class Person {
    String name = "Ravi";
    int age = 40;
}

class Employee extends Person {
    int empId = 501;
    double salary = 50000;
}

class Manager extends Employee {
    String department = "Sales";

    void display() {
        System.out.println("Name      : " + name);
        System.out.println("Age       : " + age);
        System.out.println("Emp ID    : " + empId);
        System.out.println("Salary    : " + salary);
        System.out.println("Department: " + department);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.display();
    }
}
```

3. Output

```
Name      : Ravi
Age       : 40
Emp ID    : 501
Salary    : 50000.0
Department: Sales
```

Question 4: Method Overriding

1. Question

Write a Java program to demonstrate method overriding by creating an Animal superclass with a sound() method. Create subclasses Dog and Cat that override the sound() method with their respective sounds. Display the appropriate sound for each animal.

2. Program

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks: Woof Woof");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows: Meow Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
        d.sound();
        c.sound();
    }
}
```

3. Output

```
Dog barks: Woof Woof
Cat meows: Meow Meow
```

Question 5: Dynamic Method Dispatch

1. Question

Write a Java program to demonstrate dynamic method dispatch using a superclass Shape and subclasses Circle, Rectangle, and Triangle. Override the draw() method in each subclass and use a superclass reference to invoke the appropriate overridden method at runtime.

2. Program

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;
        s = new Circle();
        s.draw();
        s = new Rectangle();
        s.draw();
        s = new Triangle();
        s.draw();
    }
}
```

3. Output

```
Drawing a Circle
Drawing a Rectangle
Drawing a Triangle
```

Question 6: super with Method Overriding

1. Question

Write a Java program to demonstrate the use of super with method overriding. Create an Employee class with a display() method and derive a Manager class that overrides the method. Use super.display() in the subclass to invoke the superclass implementation before displaying the manager details.

2. Program

```
class Employee {
    String name = "Anil";
    int id = 201;

    void display() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID  : " + id);
    }
}

class Manager extends Employee {
    String department = "HR";

    void display() {
        super.display();
        System.out.println("Department   : " + department);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.display();
    }
}
```

3. Output

```
Employee Name: Anil
Employee ID  : 201
Department   : HR
```

Question 7: Multilevel Inheritance for Salary Calculation

1. Question

Write a Java program to create a multilevel inheritance hierarchy Employee → Developer → SeniorDeveloper. Store basic salary in the Employee class, programming allowance in the Developer class, and project allowance in the SeniorDeveloper class. Calculate and display the total salary.

2. Program

```
class Employee {
    double basicSalary = 30000;
}

class Developer extends Employee {
    double programmingAllowance = 8000;
}

class SeniorDeveloper extends Developer {
    double projectAllowance = 12000;

    void display() {
        double total = basicSalary + programmingAllowance + projectAllowance;
        System.out.println("Basic Salary         : " + basicSalary);
        System.out.println("Programming Allowance: " + programmingAllowance);
        System.out.println("Project Allowance    : " + projectAllowance);
        System.out.println("Total Salary         : " + total);
    }
}

public class Main {
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.display();
    }
}
```

3. Output

```
Basic Salary         : 30000.0
Programming Allowance: 8000.0
Project Allowance    : 12000.0
Total Salary         : 50000.0
```

Question 8: Dynamic Method Dispatch for Bank Accounts

1. Question

Write a Java program to demonstrate dynamic method dispatch using a superclass BankAccount with a calculateInterest() method. Create SavingsAccount and CurrentAccount subclasses with different implementations of the method. Use a superclass reference to calculate interest for different account types.

2. Program

```
class BankAccount {
    double balance = 10000;

    void calculateInterest() {
        System.out.println("Interest not defined");
    }
}

class SavingsAccount extends BankAccount {
    void calculateInterest() {
        double interest = balance * 4 / 100;
        System.out.println("Savings Account Interest: " + interest);
    }
}

class CurrentAccount extends BankAccount {
    void calculateInterest() {
        double interest = balance * 1 / 100;
        System.out.println("Current Account Interest: " + interest);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc;
        acc = new SavingsAccount();
        acc.calculateInterest();
        acc = new CurrentAccount();
        acc.calculateInterest();
    }
}
```

3. Output

```
Savings Account Interest: 400.0
Current Account Interest: 100.0
```

Question 9: Constructor Execution in Multilevel Inheritance

1. Question

Write a Java program to demonstrate the order of constructor execution in multilevel inheritance using the classes Person, Student, and GraduateStudent. Use constructors in all three classes and display messages to show the order in which the constructors are executed.

2. Program

```
class Person {
    Person() {
        System.out.println("Person constructor called");
    }
}

class Student extends Person {
    Student() {
        System.out.println("Student constructor called");
    }
}

class GraduateStudent extends Student {
    GraduateStudent() {
        System.out.println("GraduateStudent constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        GraduateStudent g = new GraduateStudent();
    }
}
```

3. Output

```
Person constructor called
Student constructor called
GraduateStudent constructor called
```

Question 10: Banking Application Using Inheritance

1. Question

Write a Java program to develop a simple banking application using inheritance. Create a BankAccount superclass with account number and balance. Derive SavingsAccount and CurrentAccount classes and provide suitable methods for deposit, withdrawal, and interest calculation. Demonstrate inheritance and method overriding.

2. Program

```
class BankAccount {
    int accNo;
    double balance;

    BankAccount(int accNo, double balance) {
        this.accNo = accNo;
        this.balance = balance;
    }

    void deposit(double amt) {
        balance += amt;
        System.out.println("Deposited " + amt + ", Balance: " + balance);
    }

    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("Withdrawn " + amt + ", Balance: " + balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    void calculateInterest() {
        System.out.println("No interest for general account");
    }
}

class SavingsAccount extends BankAccount {
    SavingsAccount(int accNo, double balance) {
        super(accNo, balance);
    }

    void calculateInterest() {
        System.out.println("Savings Interest: " + (balance * 4 / 100));
    }
}

class CurrentAccount extends BankAccount {
    CurrentAccount(int accNo, double balance) {
        super(accNo, balance);
    }

    void calculateInterest() {
        System.out.println("Current Interest: " + (balance * 1 / 100));
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        SavingsAccount s = new SavingsAccount(1001, 5000);
        CurrentAccount c = new CurrentAccount(2001, 8000);

        System.out.println("--- Savings Account ---");
        s.deposit(2000);
        s.withdraw(1000);
        s.calculateInterest();

        System.out.println("--- Current Account ---");
        c.deposit(1500);
        c.withdraw(10000);
        c.calculateInterest();
    }
}
```

3. Output

```
--- Savings Account ---
Deposited 2000.0, Balance: 7000.0
Withdrawn 1000.0, Balance: 6000.0
Savings Interest: 240.0
--- Current Account ---
Deposited 1500.0, Balance: 9500.0
Insufficient balance
Current Interest: 95.0
```